

UNIVERSIDAD EAFIT
ESCUELA DE CIENCIAS APLICADAS E INGENIERÍA
ARQUITECTURA DE SOFTWARE 2026

Tutorial 04: Dockerización y Despliegue en AWS

De Localhost a la Nube: Infraestructura Inmutable

Nicolás Rodríguez

Profesor: Nicolás Ramírez Vélez

30 de septiembre de 2026

1. Evidencia de entrega: API servida desde AWS EC2

Aplicación desplegada en una instancia EC2 del AWS Academy Learner Lab (Amazon Linux 2023, t2.micro, IP pública 13.222.171.42) con `docker compose up -d --build`, y el puerto 8000 abierto en el Security Group:

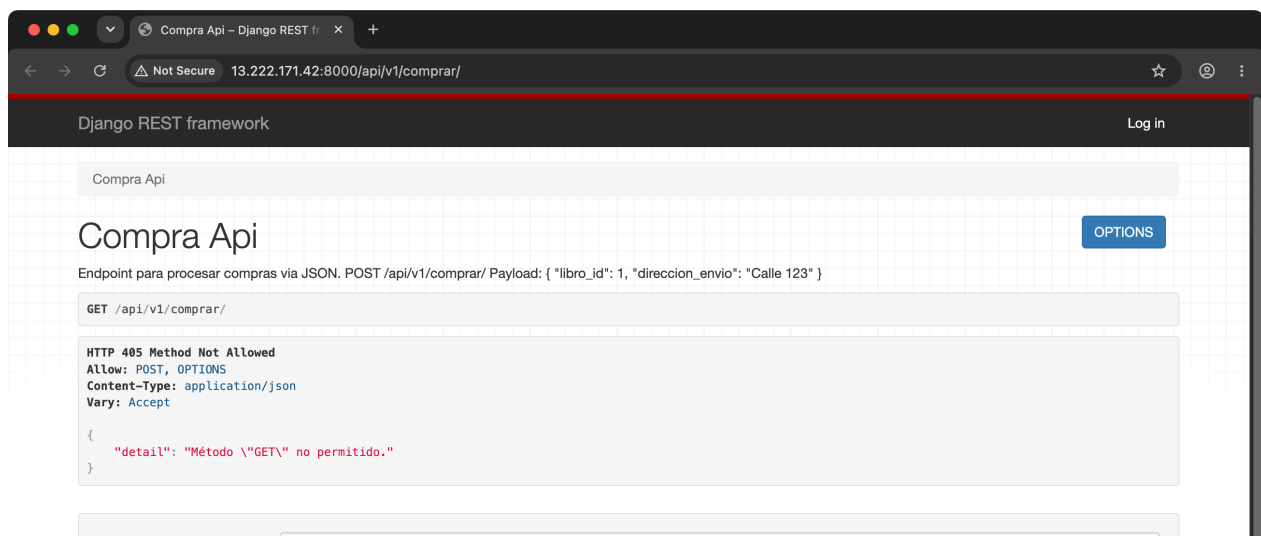


Figura 1: `http://13.222.171.42:8000/api/v1/comprar/` — respuesta JSON de la API servida desde AWS EC2.

2. Paso 1: Configuración desde el ambiente (`settings.py`)

Las credenciales de la base de datos ya no están quemadas: se leen con `os.environ`. Una decisión propia: si no existe `DB_HOST`, el proyecto cae a SQLite para que `python manage.py runserver` y los tests sigan funcionando en local sin montar un Postgres. En Docker y en AWS la variable siempre existe, así que corre sobre **PostgreSQL 15**.

Tienda/`settings.py`

```
# Tutorial 04: la configuracion viene del ambiente, no del codigo.  
# En Docker/AWS (DB_HOST definido) usamos PostgreSQL; sin variables de  
# entorno, el proyecto sigue corriendo en local con SQLite.
```

```

if os.environ.get('DB_HOST'):
    DATABASES = {
        'default': {
            'ENGINE': 'django.db.backends.postgresql',
            'NAME': os.environ.get('DB_NAME', 'tienda_db'),
            'USER': os.environ.get('DB_USER', 'postgres'),
            'PASSWORD': os.environ.get('DB_PASSWORD', 'postgres'),
            'HOST': os.environ.get('DB_HOST', 'localhost'), # Importante para Docker
            'PORT': os.environ.get('DB_PORT', '5432'),
        }
    }
else:
    DATABASES = {
        'default': {
            'ENGINE': 'django.db.backends.sqlite3',
            'NAME': BASE_DIR / 'db.sqlite3',
        }
    }

```

requirements.txt (pip freeze - incluye psycopg2-binary y djangorestframework)

```

asgiref==3.12.1
Django==5.2.17
django-filter==26.1
djangorestframework==3.18.1
gunicorn==26.2.0
Markdown==3.11
psycopg2-binary==2.9.13
sqlparse==0.6.0
whitenoise==6.12.0

```

3. Paso 2: Dockerfile

Dockerfile

```

# 1. Usamos una imagen base oficial de Python ligera
FROM python:3.11-slim

# 2. Evitamos que Python escriba archivos .pyc y buffer de logs
ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

# 3. Establecemos el directorio de trabajo dentro del contenedor
WORKDIR /app

# 4. Copiamos los requerimientos e instalamos dependencias
COPY requirements.txt /app/
RUN pip install --no-cache-dir -r requirements.txt

# 5. Copiamos el resto del código fuente
COPY . /app/

# 6. Comando por defecto al iniciar el contenedor
# Escucha en el puerto 8000 disponible para el mundo (0.0.0.0)
CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]

```

Complementé con un `.dockerignore` (excluye `.venv`, `.git`, `db.sqlite3` y logs) para que la imagen no cargue basura ni secretos locales.

4. Paso 3: Orquestación local con Docker Compose

`docker-compose.yml`

```
services:
  db:
    image: postgres:15
    volumes:
      - postgres_data:/var/lib/postgresql/data
    environment:
      - POSTGRES_DB=tienda_db
      - POSTGRES_USER=postgres
      - POSTGRES_PASSWORD=secret_password
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d tienda_db"]
      interval: 3s
      timeout: 3s
      retries: 20

  web_django: # El Monolito (v1)
    build: .
    command: sh -c "python manage.py migrate && python manage.py seed_tienda && python manage.py collectstatic --noinput && gunicorn Tienda.wsgi:application --bind 0.0.0.0:8000"
    volumes:
      - ./app
    depends_on:
      db:
        condition: service_healthy
    environment:
      - DB_NAME=tienda_db
      - DB_USER=postgres
      - DB_PASSWORD=secret_password
      - DB_HOST=db
      - DB_PORT=5432

  pagos_flask: # El Microservicio (v2) - EL ESTRANGULADOR
    build: ./microservicio_pagos
    ports:
      - "5000:5000" # Solo para pruebas iniciales, luego lo cerraremos

  nginx:
    image: nginx:1.25
    ports:
      - "80:80"
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf
    depends_on:
      - web_django
      - pagos_flask

volumes:
  postgres_data:
```

Tres ajustes sobre el enunciado, todos a propósito: (1) quité `version: '3.8'` porque Compose v2

lo ignora y lanza advertencia; (2) agregué un **healthcheck** a Postgres y **depends_on: condition: service_healthy**, porque **depends_on** a secas solo espera a que el contenedor arranque, no a que la base acepte conexiones; (3) el **command** ejecuta **migrate** y **seed_tienda** antes del servidor, así una instancia recién creada queda con tablas y catálogo sin pasos manuales.

4.1. Prueba local: docker compose up --build

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
proyectotienda-tutoriales-db-1	postgres:15	"docker-entrypoint.s..."	db	About a minute ago	Up About a minute (healthy)	5432/tcp
proyectotienda-tutoriales-web-1	proyectotienda-tutoriales-web	"sh -c 'python manag..."	web	About a minute ago	Up 57 seconds	0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp

w id	titulo	total	direccion_envio			

1	Clean Architecture	214.20	Cra 49 #7 Sur-50, Medellín			
(1 row)						

Figura 2: docker compose ps: contenedores db (postgres:15, healthy) y web arriba; la orden creada vía API ya vive en PostgreSQL.

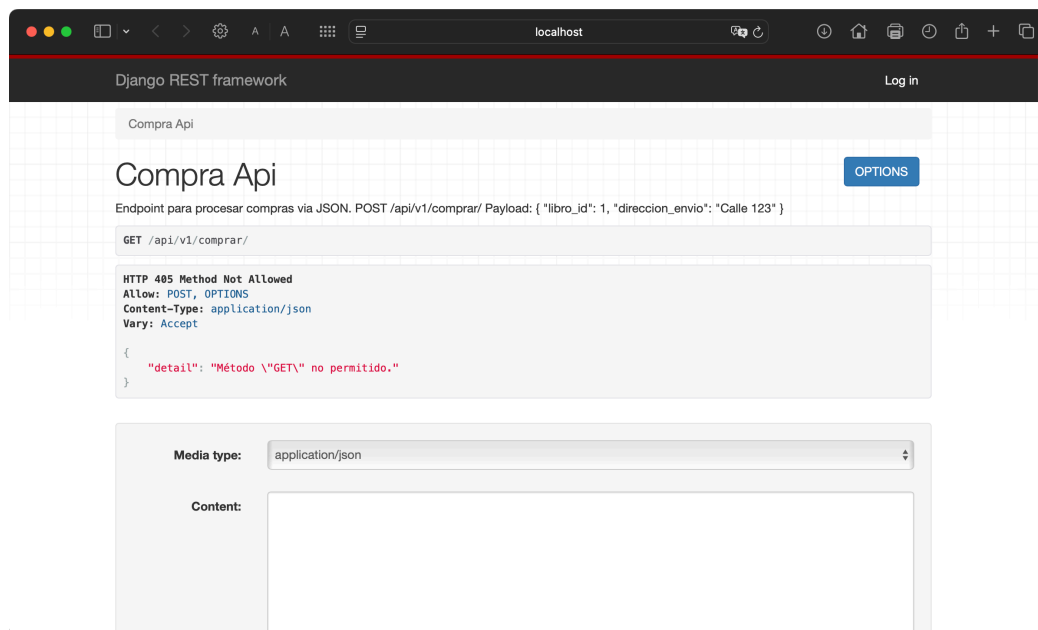


Figura 3: localhost:8000/api/v1/comprar/ servido desde el contenedor: respuesta JSON de la API (GET no está permitido, el endpoint solo acepta POST).

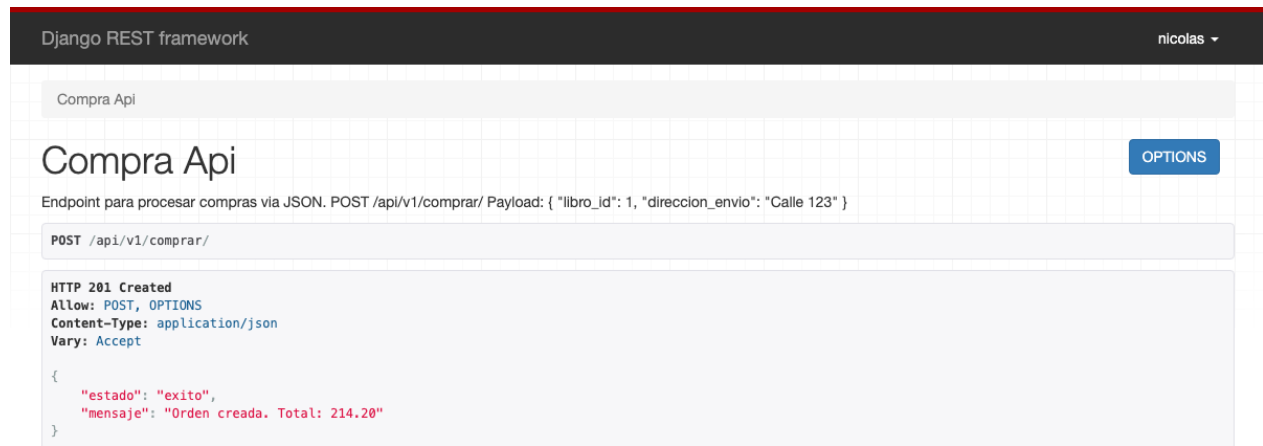


Figura 4: POST de compra contra el contenedor: HTTP 201 Created, orden persistida en PostgreSQL.

5. Paso 4: Despliegue en AWS Academy

1. AWS Academy → Learner Lab → **Start Lab** → consola **EC2** → **Launch Instance**.
2. Name: **Servidor-Django** · AMI: **Amazon Linux 2023** · Tipo: **t2.micro** · Key pair: **vockey**.
3. Network settings: marcar **Allow SSH**, **Allow HTTP** y **Allow HTTPS** → Launch.
4. Instancia → **Connect** → pestaña **EC2 Instance Connect** → Connect.

En la terminal de la instancia ejecuté los mismos comandos del tutorial. La única diferencia: el código lo subí directamente desde mi equipo con `scp` (en lugar de `git clone`), porque el proyecto no está publicado en un repositorio de GitHub:

Terminal EC2

```
# Paso 4.3 - preparar la instancia
sudo dnf update -y
sudo dnf install git docker -y
sudo service docker start
sudo usermod -a -G docker ec2-user
exit # reconectar para aplicar permisos

# Paso 4.4 - traer el código y desplegar
scp -i llave.pem codigo.tgz ec2-user@<IP-EC2>:/tmp/ # (desde mi equipo)
mkdir -p ~/tienda-arquitectura-2026 && cd ~/tienda-arquitectura-2026
tar -xzf /tmp/codigo.tgz
docker compose up -d --build
docker ps
```

5.1. Abrir el puerto 8000 (Security Group)

1. EC2 → Instances → seleccionar la instancia → pestaña **Security**.
2. Clic en el Security Group (ej. `launch-wizard-1`) → **Edit inbound rules** → **Add rule**.
3. Type: **Custom TCP** · Port range: **8000** · Source: **0.0.0.0/0** → **Save rules**.

6. Cómo se hizo el despliegue

Lancé la instancia desde el Learner Lab (región us-east-1) con los mismos parámetros del tutorial: Amazon Linux 2023, t2.micro, SSH/HTTP/HTTPS abiertos y regla Custom TCP 8000. Dentro de la instancia instalé Git, Docker y el plugin Compose, subí el código y levanté la arquitectura. La captura de la sección 1 es el navegador de mi equipo consumiendo la IP pública de EC2. Al terminar, la instancia se **terminó** para no consumir presupuesto del laboratorio.

Anexo: guía complementaria 04-B (referencia)

La guía 04-B es una demostración de clase (despliegue del proyecto de ejemplo Nram94/djangoDocker contra una base **MySQL 8.0 en RDS**) y no hace parte de los entregables de este tutorial. Resumen del flujo: crear la BD en RDS (Free tier), permitir el puerto 3306 desde la instancia EC2 en su Security Group, crear la base con `mysql -h <endpoint> -P 3306 -u admin -p`, configurar las credenciales y correr `docker image build / docker container run`. Lección clave: separar la base de datos (servicio gestionado) del contenedor de la aplicación, y **borrar la base y detener los contenedores** al terminar para no gastar créditos.